



MODUL 12 · AUTHENTICATION & AUTHORIZATION

CVE-2025-29927: Jangan Andalkan Middleware Saja

Vulnerability kritis yang mengubah cara berpikir tentang security di Next.js — untuk selamanya.



CVE-2025-29927 — Middleware Authorization

Bypass

CVSS Score: 9.1 (Critical). Disclosed 21 Maret 2025. Affects Next.js 11.1.4 – 15.2.2.

● ● ● Cara exploit — satu header, bypass semua!

```
# Exploit CVE-2025-29927
# Attacker cukup tambahkan satu header:
# x-middleware-subrequest: middleware

# Contoh request:
GET /admin/dashboard HTTP/1.1
Host: vulnerable-site.com
x-middleware-subrequest: middleware

# Hasil: middleware DILEWATI SEPENUHNYA!
# Tidak ada auth check, tidak ada redirect
# /admin/dashboard terbuka untuk siapapun!

# Kenapa bisa terjadi?
# Next.js gunakan header internal ini untuk
# mencegah infinite loop di middleware.
# Jika header ini ada → middleware di-skip.
# Attacker memanfaatkan header internal ini!

# Affected versions:
# 11.1.4 - 15.2.2 (SEMUA!)
# Fixed: 15.2.3, 14.2.25, 13.5.9, 12.3.5
# Next.js 16+: FIXED (gunakan proxy.ts)
```

Dampak

Aplikasi yang menggunakan middleware/proxy SEBAGAI SATU-SATUNYA auth check:

- ✗ Admin panel bisa diakses tanpa login
- ✗ User data bisa dicuri
- ✗ Privilege escalation (akses admin)
- ✗ API endpoints tidak terproteksi
- ✗ Bypass Content Security Policy

Siapa yang terdampak:

Semua Next.js apps yang:

1. Pakai middleware untuk auth
2. Self-hosted (bukan Vercel managed)
3. Belum di-patch ke v15.2.3+

Next.js 16: SUDAH FIXED
tapi pelajarannya tetap berlaku!

Defense in Depth — Banyak Layer Security

Satu layer gagal → layer lain masih melindungi. Ini adalah prinsip security yang benar.

● ● ● Multi-layer security — cara yang benar

```
// ❌ SALAH: hanya andalkan proxy.ts
// proxy.ts → check session → ok
// app/dashboard/page.tsx → langsung render (tidak check!)
// Jika proxy dibypass → user masuk tanpa auth!

// ✅ BENAR: cek di setiap layer

// — Layer 1: proxy.ts (gating/UX) —————
// Redirect user yang belum login → /login
// UX yang baik: user tidak lihat halaman kosong

// — Layer 2: Server Component (actual security) —————
export default async function DashboardPage() {
  const session = await auth.api.getSession({ headers: await headers() })
  if (!session) redirect("/login") // ← WAJIB ADA!
  // data fetching hanya jika session valid
  const data = await db.post.findMany({ where: { authorId: session.user.id } })
  return <Dashboard data={data} />
}

// — Layer 3: Server Action (data mutation) —————
export async function deletePost(postId: string) {
  "use server"
  const session = await auth.api.getSession({ headers: await headers() })
  if (!session) return { error: "Unauthorized" } // ← WAJIB ADA!
  // verify ownership + delete
}

// — Layer 4: Data layer (query-level security) —————
// Selalu filter by userId agar satu user tidak bisa akses data user lain:
const posts = await db.post.findMany({
  where: { authorId: session.user.id } // ← selalu include userId!
})
```

Security Checklist — CVE-2025-29927 Prevention

Empat hal yang harus dilakukan untuk memastikan aplikasi Next.js aman.



Update Next.js ke v16+

Next.js 16 sudah fix CVE-2025-29927. Jika project lama: update ke 15.2.3, 14.2.25, atau 13.5.9 minimal.



Cek Session di Setiap Page

Setiap protected page.tsx dan layout.tsx harus `auth.api.getSession()`. Jangan assume proxy sudah cukup.



Cek Session di Setiap Action

Setiap Server Action yang mutasi data harus `getSession()`. Auth check adalah baris pertama fungsi.



Filter Data dengan userId

Selalu `WHERE authorId = session.user.id` di query. Ini mencegah IDOR — user A tidak bisa akses data user B.

Evaluasi Pemahaman

Jawab sebelum lanjut.

Q1 — CVE-2025-29927 dieksploitasi dengan cara apa?

- A) SQL injection ke database B) Tambahkan header x-middleware-subrequest: middleware → middleware di-skip sepenuhnya C) XSS di login form

Q2 — Jika proxy.ts bisa dibypass, layer apa yang masih melindungi route yang aman?

- A) Tidak ada — harus update Next.js dulu B) Session check di Server Component dan Server Action (defense in depth) C) CORS headers di response

Q3 — Mengapa query filter 'WHERE authorId = session.user.id' penting meski sudah ada auth check?

- A) Untuk performa query yang lebih cepat B) Mencegah IDOR — user A tidak bisa akses/hapus data milik user B meskipun sudah login C) Required oleh Prisma 7

Tugas Mandiri

Audit project untuk CVE-2025-29927 dan implementasikan defense in depth.

01 Version Audit

Cek versi Next.js di package.json. Jika < 15.2.3 → update: bun update next react react-dom. Verifikasi setelah update.

02 Page Audit

Review semua protected pages. Pastikan setiap page.tsx di /dashboard, /admin, /profile: punya session check + redirect jika tidak ada session.

03 Action Audit

Review semua Server Actions di actions/. Pastikan setiap fungsi yang mutasi data: session check adalah baris pertama.

04 Data Filter Audit

Review semua db queries. Pastikan setiap query yang return user-specific data: ada WHERE userId = session.user.id atau filter equivalent.

 Prinsip: Never trust the edge. Proxy/middleware = UX only. Actual security ada di data layer.

Yang Sudah Kita Pelajari

- ✓ CVE-2025-29927: attacker tambah header x-middleware-subrequest → middleware di-skip. CVSS 9.1.
- ✓ Fix: Next.js v16+ sudah patch. Jika v15: update ke 15.2.3+.
- ✓ Pelajaran utama: JANGAN andalkan middleware/proxy saja sebagai satu-satunya security.
- ✓ Defense in depth: proxy.ts + Server Component check + Server Action check + data filter.
- ✓ Selalu WHERE authorId = session.user.id di query (mencegah IDOR).

→ Selanjutnya: **Bab 6 — RBAC: Role-Based Access Control**

